Reconnaissance des périphériques et prise de contrôle de l'écran sous Windows

Homework du Pr BELKHIR Abdelkader · Remise : 30 octobre 2026

BENALI Amel ·

1. Introduction

Ce travail répond aux deux demandes du sujet : (1) un programme qui reconnaît tous les périphériques d'une machine et édite les caractéristiques de chacun, (2) la prise de contrôle d'un périphérique, ici l'**écran**, sous Windows. Le programme a été développé sur Windows 11 (build 26200) avec Python 3.14, sur un Dell Inspiron 14 7425 (AMD Ryzen 5 5625U, 16 Go de RAM, SSD NVMe de 238,5 Go, carte graphique AMD Radeon).

2. Choix techniques

Python permet d'interroger Windows en peu de lignes, avec un code lisible et rapide à tester. Outils utilisés :

- `wmi` (avec `pywin32`) : interroge WMI, l'interface de Windows qui expose le matériel.
- `ctypes` : appelle directement l'API Windows (`user32.dll`, `kernel32.dll`) pour piloter l'écran, la souris et le clavier.
- `Pillow` : capture d'écran.
- PowerShell et `powercfg`, lancés avec `subprocess` : radios Wi-Fi et Bluetooth, économiseur de batterie.

Les dépendances (`wmi`, `pywin32`, `pillow`) sont listées dans `requirements.txt` et s'installent dans un environnement virtuel.

3. Architecture

Fichier	Rôle
`main.py`	Menu principal (liste, rapport, contrôle de l'écran) et gestion des erreurs
`reconnaissance.py`	Partie 1 : liste des périphériques, caractéristiques détaillées, rapport TXT

Fichier	Rôle
`controle_ecran.py`	Partie 2 : prise de contrôle de l'écran (menu de 13 actions)
`requirements.txt`, `README.md`	Dépendances ; installation et utilisation

`main.py` appelle les deux modules, qui peuvent aussi être lancés seuls.

4. Partie 1 : reconnaissance des périphériques

4.1 Détection de tous les périphériques

`reconnaissance.py` interroge la classe WMI `Win32_PnPEntity`, qui recense tous les périphériques Plug and Play. Pour chacun, il relève le nom, la catégorie (`PNPClass`), le fabricant et l'état. Sur la machine de test, **179 périphériques répartis en 30 catégories** sont détectés (audio, batterie, Bluetooth, caméra, disque, écran, empreinte digitale, firmware, etc.).

4.2 Caractéristiques détaillées

Des classes WMI spécialisées donnent les informations précises de chaque type de matériel :

Catégorie	Classe WMI	Informations relevées
Processeur	`Win32_Processor`	Nom, fabricant, coeurs, threads, fréquence
Mémoire	`Win32_PhysicalMemory`	Capacité, vitesse, fabricant
Disques	`Win32_DiskDrive`	Modèle, capacité, interface, partitions, état
Carte graphique	`Win32_VideoController`	Nom, driver, résolution, fréquence
Réseau	`Win32_NetworkAdapter`	Nom, fabricant, adresse MAC, connexion
Audio	`Win32_SoundDevice`	Nom, fabricant, état
Batterie	`Win32_Battery`	Nom, charge, statut
Clavier, souris	`Win32_Keyboard`, `Win32_PointingDevice`	Nom, disposition, état

Catégorie	Classe WMI	Informations relevées
USB, imprimantes, BIOS	`Win32_USBController`, `Win32_Printer`, `Win32_BIOS`	Nom, driver, version, date

Une propriété absente s'affiche `N/A`, car tous les périphériques ne fournissent pas toutes les informations.

4.3 Mise en forme du rapport

- Les codes sont traduits : statut de batterie `2` devient « Sur secteur », disposition clavier `0000040C` devient « Français (France) ».
- La date du BIOS est reformatée (`23/04/2025`).
- Les éléments identiques sont regroupés avec une colonne Quantité (3 souris HID, 2 barrettes de RAM).
- Les adaptateurs réseau virtuels sans connexion sont masqués, avec une note indiquant leur nombre.

Le rapport s'affiche dans le terminal et s'enregistre dans `rapport_peripheriques.txt`.

4.4 Remarques sur les résultats

- Le disque NVMe apparaît avec l'interface « SCSI » : c'est ainsi que Windows présente les disques NVMe.
- La propriété `NetEnabled` de WMI indique si l'adaptateur est **connecté**, pas s'il est activé : la colonne s'intitule « Connecté ».
- Le champ fabricant des contrôleurs USB contient le nom générique du driver : c'est une particularité de WMI.

5. Partie 2 : prise de contrôle de l'écran

5.1 Actions réalisées

Le sujet demande de prendre le contrôle de l'écran. Le module `controle_ecran.py` regroupe 13 actions accessibles par un menu. Elles s'appuient sur l'API Windows (via `ctypes`), sur WMI et sur PowerShell.

N°	Action	Technique
1	Informations : nombre d'écrans, résolution, fréquence, luminosité	WMI `Win32_VideoController`, `GetSystemMetrics`

N°	Action	Technique
2	Luminosité : lire, +10 %, -10 %, valeur précise, démonstration progressive	WMI `WmiMonitorBrightness` et `WmiMonitorBrightnessMethods`
3	Résolution : liste des modes, changement temporaire	`EnumDisplaySettingsW`, `ChangeDisplaySettingsW`
4	Capture d'écran enregistrée en PNG	`Pillow` (`ImageGrab`)
5	Souris : déplacement fluide du curseur (rectangle)	`GetCursorPos`, `SetCursorPos`
6	Clavier : ouverture du Bloc-notes et saisie automatique	`SendInput`
7	Mise en plein écran de la fenêtre active	`GetForegroundWindow`, `ShowWindow`
8	Affichage du bureau (Windows + D)	`keybd_event`
9	Verrouillage de la session	`LockWorkStation`
10	Extinction de l'écran	`WM_SYSCOMMAND`, `SC_MONITORPOWER`
11	Changement de la langue de saisie du clavier	`GetKeyboardLayoutList`, `WM_INPUTLANGCHANGEREQUEST`
12	Son : volume, muet, test sonore	touches multimédia (`keybd_event`), `winsound`
13	Wi-Fi, Bluetooth, mode avion, économiseur de batterie	API Windows Radios via PowerShell, `powercfg`

5.2 Précautions

- Les réglages temporaires se rétablissent seuls : la résolution revient à sa valeur d'origine après 8 secondes, et la luminosité d'origine est restaurée à la fin de la démonstration (bloc `finally`).
- Les actions sensibles demandent une confirmation : verrouillage de la session, désactivation du Wi-Fi, mode avion.
- Les valeurs saisies sont contrôlées : luminosité bornée entre 0 et 100, numéros de menu validés.

5.3 Limites connues

- Le réglage logiciel de la luminosité fonctionne sur un écran intégré (portable). Un écran externe peut ne pas le permettre : le menu affiche alors « non disponible ».

- Windows change le volume de 2 % par appui de touche : le réglage à une valeur précise est approximatif.
- Le mode avion est reproduit en coupant toutes les radios, et l'économiseur de batterie est piloté par son seuil d'activation automatique.
- Le changement de langue de saisie s'applique à la fenêtre active.

6. Gestion des erreurs

- Saisie invalide dans un menu : message « Choix invalide » et retour au menu.
- Fonction indisponible (luminosité non gérée, une seule langue installée, radio absente) : message explicite, sans arrêt du programme.
- Toute exception pendant une action est interceptée par le menu, et `safe_run()` dans `main.py` protège la liste, le rapport et le contrôle de l'écran.
- Une interruption clavier (Ctrl+C) quitte proprement.

7. Tests

Test	Résultat attendu	Résultat
Saisie invalide dans le menu	« Choix invalide », le menu revient	à compléter
Option 1 : liste des périphériques	Environ 179 périphériques, 30 catégories	à compléter
Option 2 : rapport détaillé	Tableaux affichés, fichier TXT créé	à compléter
Écran, option 1 : informations	1 écran, Radeon, 1920 x 1200, 60 Hz	à compléter
Écran, option 2 : luminosité	La luminosité change à l'écran puis revient	à compléter
Écran, option 3 : résolution	L'écran change pendant 8 s puis revient seul	à compléter
Écran, option 4 : capture	Un PNG apparaît dans `captures_ecran`	à compléter
Écran, options 5 à 8 : souris, clavier, plein écran, bureau	Rectangle du curseur, texte saisi dans le Bloc-notes, fenêtre maximisée, bureau affiché	à compléter

Test	Résultat attendu	Résultat
Écran, options 11 et 12 : langue et son	Indicateur de langue modifié, volume et notes audibles	à compléter
Écran, option 13 : radios et économiseur	État du Bluetooth et de l'économiseur modifié puis rétabli	à compléter

8. Conclusion

Le programme répond aux deux demandes : il reconnaît l'ensemble des périphériques de la machine avec leurs caractéristiques (179 périphériques, 30 catégories, détails par type de matériel) et il prend le contrôle de l'écran par un menu de 13 actions : luminosité, résolution, capture, souris, clavier, fenêtres, verrouillage, extinction, langue, son et connectivité. Le code, découpé en trois fichiers, reste simple à lire et à étendre, par exemple vers une interface graphique ou la prise en charge des écrans externes.